

GitHub guide for Akash (maintainer)

Project: SBP Wall pyRevit extension · **Repo name:** SBP-pyRevit (private) · **Owner:** your personal GitHub account

This guide is for **you** (you build and publish the tool). Your colleagues get a separate, simpler guide: **INSTALL.md**.

1. The idea in one picture



- **Commit** = save a snapshot with a short note ("fix invisible line").
- **Push** = upload your commits to GitHub.
- **Pull** = download the latest version (colleagues do this to update).
- **Branch** = a separate line of work. **main** is what colleagues use and must always work. **dev** is where you and Claude build new things.
- **Tag / Release** = a named version (v1.0, v2.0) with notes.

2. One-time setup (about 30 minutes)

2.1 Accounts and tools

1. Create or sign in to a **GitHub account**: <https://github.com>
2. Install **GitHub Desktop**: <https://desktop.github.com>. Sign in with your account.
3. Git is already on your PC (your `git status` command ran). Optional but useful: install the **GitHub CLI** (<https://cli.github.com>), then run `gh auth login` once in the VS Code terminal. It lets Claude Code create the repo and releases for you.

2.2 Create the repo folder (let Claude Code do it)

Paste this into Claude Code in VS Code:

```
Set up a Git repo for the SBP extension. Do NOT change any code.
1. Create D:\Dev\SBP-pyRevit with this layout (copy from COMPANY SET UP):
   SBP.extension\ (lib, SBP.tab)  docs\ (HISTORY.md, ADR, ref\, PNGs, PDF)
```

```

    preview\    tests\ (if any)    CLAUDE.md    README.md    CHANGELOG.md    INSTALL.md
.gitignore
    Use the .gitignore, CHANGELOG.md and INSTALL.md from COMPANY SET UP\github.
2. Update the paths inside CLAUDE.md and HISTORY.md to the new layout.
3. git init, first commit "v1.0 - first working version (tested in Revit 2026)", branch
main, tag v1.0.
4. Create a dev branch from main.
5. If gh is logged in: create a PRIVATE GitHub repo SBP-pyRevit, push main, dev and the
tag,
    and create a GitHub Release v1.0 from CHANGELOG.md.
6. Tell me what to change in pyRevit so Revit loads the extension from D:\Dev\SBP-
pyRevit.
Keep COMPANY SET UP as it is (don't delete) until I confirm.

```

Why move out of OneDrive? OneDrive and Git both try to sync the same files and can corrupt the hidden `.git` folder. GitHub is now your backup, so OneDrive isn't needed for this folder.

2.3 Point pyRevit to the new folder

pyRevit → **Settings** → **Custom Extension Directories:** - **Remove** `D:\01_SP-\OneDrive - IC Singapore\COMPANY SET UP\pyRevit` - **Add** `D:\Dev\SBP-pyRevit` - **Save Settings and Reload**, then check that the SBP tab appears **once**.

2.4 Give colleagues access

GitHub → your repo → **Settings** → **Collaborators** → **Add people** → type their GitHub username or email. They get an email invite and must **accept** it before they can install.

⚠ Important: on a *personal* account, every collaborator can also **push** (change) the code. To give colleagues **read-only** access, create a free **GitHub Organization** (for example `icsg-bim`), move the repo into it (Settings → Transfer), and add colleagues with the **Read** role. Recommended once more than 2-3 people use it.

3. Everyday work (on the dev branch)

In VS Code, the bottom-left corner shows the current branch. It should say **dev** while you're working.

You want to...	Ask Claude Code	Or do it yourself in VS Code
Save a snapshot	Commit my changes with a short clear message	Source Control (Ctrl+Shift+G) → type message → Commit
Upload to GitHub	Push dev to GitHub	Source Control → Sync Changes

You want to...	Ask Claude Code	Or do it yourself in VS Code
See what changed	Show me what changed since the last commit	Click a file in Source Control → side-by-side diff
See history	Show the history of script.py	Timeline panel (bottom left)
Undo a bad change	Restore script.py to the last commit	Source Control → right-click file → Discard Changes

Good habits: - Commit **small and often**, one idea per commit (e.g. "add web rule field"). - Always test in Revit before a release. - Never commit passwords, tokens or project/client data.

4. Publishing a new version to colleagues (release)

When dev works and is tested in Revit:

Release a new version:

1. Update CHANGELOG.md with what changed (plain words, for colleagues).
2. Merge dev into main.
3. Tag it (e.g. v2.0) and push main, dev and the tag.
4. Create a GitHub Release with the CHANGELOG notes.
5. Switch back to dev.

Then tell your colleagues: **"New SBP version v2.0. Open GitHub Desktop → Pull → pyRevit Reload."**

Version numbers: - **v1.0 → v1.1**: a bug fix, same way of working. - **v1.1 → v2.0**: new features or changed behaviour (like the new corner and web rules).

5. If something goes wrong

Problem	What to do
A release is broken for colleagues	Ask Claude Code: Revert main to tag v1.0 and push . Colleagues pull again and are back on the old version.
You edited on main by mistake	Move my uncommitted changes from main to dev
"Push rejected"	Pull first, then push again (someone, or you on another PC, pushed first)
Two SBP tabs in Revit	An old folder is still in the pyRevit extension list. Remove it in Settings.
You want an old version of a file	Show me script.py as it was in v1.0

6. Checklist

- ☐ GitHub account + GitHub Desktop (+ optional gh CLI)
- ☐ Repo folder created at `D:\Dev\SBP-pyRevit` , first commit, tag v1.0
- ☐ Private repo on GitHub, main + dev pushed
- ☐ pyRevit points to the new folder, one SBP tab
- ☐ Colleagues invited (or Organization with Read role)
- ☐ INSTALL.md sent to colleagues